

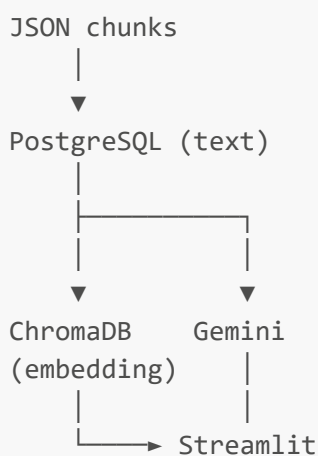
# BÀI THỰC HÀNH 6 — Xây dựng RAG với AI Agent

## 1. Mục tiêu

Trong bài thực hành này, bạn sẽ sử dụng AI Agent để xây dựng một hệ thống RAG nhỏ chạy trên máy cá nhân.

- Không cần viết nhiều code bằng tay.
- Mục tiêu là học cách giao việc cho AI Agent và hiểu luồng hoạt động của RAG.

## 2. Sơ đồ luồng dữ liệu



## 3. Dữ liệu sử dụng

Tái sử dụng từ Buổi 5 tại đường dẫn: [RAG/rag\\_foundation/buoi\\_05/](#)

```
RAG/rag_foundation/buoi_05/  
├── .venv/  
└── output/chunks/
```

- ❌ Không OCR.
- ❌ Không xử lý PDF.
- ❌ Không chunk lại.
- ❌ Không sửa Buổi 5 (Buổi 5 chỉ được xem là nguồn dữ liệu đầu vào).

## 4. Mục tiêu ứng dụng

Ứng dụng hoàn chỉnh có thể:

1. Đọc JSON.

2. Index dữ liệu.
3. Hỏi đáp bằng Gemini.
4. Hiển thị kết quả trên Streamlit.

(Không cần thêm các tính năng production).

---

## 5. Các bước thực hiện

### Bước 1 — Tạo project

Gửi prompt sau cho AI Agent:

```
[GOAL]
Tạo project mới tại đường dẫn: `RAG/rag_foundation/bui_06/` tại trong
Rag_thuchanh\RAG\rag_foundation

Đây là project demo phục vụ workshop. Chỉ cần tạo các file, chưa cần chứa nội
dung code:

- `app.py`
- `rag.py`
- `requirements.txt`
- `.env.example`
- `README.md`

Không tạo:

- `tests`
- `docs`
- `CLI`
- `logging`
- nhiều module phụ

Ưu tiên project nhỏ, dễ đọc.
```

---

### Bước 2 — Tạo Agent Specification

Trước khi viết code, tạo file `SPEC_bui_06.md`. Gửi prompt sau cho AI Agent:

[GOAL]

Tạo file `SPEC_bui_06.md`. Đây là tài liệu hướng dẫn AI Agent. Nguồn nội dung như sau:

```
## Workspace
Chỉ được phép đọc:
- RAG/rag_foundation/bui_05/output/chunks/
- RAG/rag_foundation/bui_05/.env/
- RAG/rag_foundation/bui_06/
```

Không đọc:

- source code của Buổi 5
- README các buổi trước
- notebook
- git history
- các thư mục khác

Buổi 5 là black box. Không reverse engineering. Không phân tích cách Buổi 5 hoạt động.

## ## Python

Sử dụng đúng interpreter trong: `RAG/rag\_foundation/buoi\_05/.venv/`  
Không tạo virtual environment mới.

## ## Package

Chỉ cài:

- streamlit
- google-genai
- chromadb
- psycpg
- python-dotenv

Không cài framework khác.

## ## Coding Style

Ưu tiên: ít file, ít class, ít function, code dễ đọc.

Không tạo: repository pattern, service layer, dependency injection, factory, plugin.

## ## Scope

Chỉ cần: index, retrieval, answer, streamlit.

Không phát triển ngoài yêu cầu.

## ## Error Handling

Chỉ cần try/except tối thiểu.

Không cần: retry, logging, monitoring.

## ## Security

Không in: API Key, password, secret.

## ## Code Size

Mục tiêu khoảng 300–500 dòng Python.

Nếu vượt khoảng 700 dòng, hãy đơn giản hóa thiết kế.

---

## Bước 3 — Chuẩn bị môi trường

Gửi prompt sau cho AI Agent:

[CONTEXT] Đọc agent spec tại file SPEC\_bui\_06.md  
[GOAL]

Chuẩn bị toàn bộ môi trường để chạy project RAG.

Đây là workshop dành cho người mới.

Ưu tiên tự động hóa tối đa, giảm thao tác thủ công.

---

[WORKSPACE]

Chỉ được phép thao tác trong:

- RAG/rag\_foundation/bui\_06/
- RAG/rag\_foundation/bui\_05/.venv/
- RAG/rag\_foundation/bui\_05/output/chunks/

Không đọc source code của các buổi trước.

---

[PYTHON]

Sử dụng đúng Python interpreter trong:

RAG/rag\_foundation/bui\_05/.venv/

Không tạo virtual environment mới.

---

[.ENV]

Nếu chưa có `.env`:

- tạo từ `.env.example`

Nếu thiếu các biến sau thì tự động thêm:

```
GEMINI_API_KEY=  
POSTGRES_HOST=localhost  
POSTGRES_PORT=5432  
POSTGRES_DB=rag_db  
POSTGRES_USER=postgres  
POSTGRES_PASSWORD=
```

Không ghi đè giá trị đã tồn tại.

---

## [PACKAGE]

Kiểm tra và cài nếu còn thiếu:

- streamlit
- google-genai
- chromadb
- psycopg
- python-dotenv

Sau khi cài đặt:

- import thử từng package
- báo PASS hoặc FAIL

---

## [CHROMADB]

Ưu tiên:

1. Nếu phát hiện Chroma Server đang chạy thì sử dụng.
2. Nếu không có Chroma Server:

tự động sử dụng Embedded Persistent Client.

Lưu dữ liệu tại:

storage/chroma/

Không yêu cầu người dùng cài đặt ChromaDB Server.

---

## [POSTGRESQL]

Mục tiêu là có một database tên:

rag\_db

Thực hiện theo thứ tự:

1. Kiểm tra PostgreSQL đã được cài đặt hay chưa.
2. Nếu chưa cài:

Dừng lại và hướng dẫn người dùng:

- tải PostgreSQL từ trang chính thức
- cài đặt
- ghi nhớ mật khẩu user postgres
- điền mật khẩu đó vào POSTGRES\_PASSWORD trong `.env`

Không tự cài PostgreSQL.

3. Sau khi PostgreSQL đã hoạt động:

kết nối tới database mặc định:

postgres

bằng:

host = localhost

port = 5432

user = postgres

password = đọc từ `.env`

4. Kiểm tra database:

rag\_db

5. Nếu chưa tồn tại:

tự động thực hiện:

CREATE DATABASE rag\_db;

6. Đóng kết nối.

7. Kết nối lại tới:

rag\_db

8. Không tạo thêm user mới.

9. Không yêu cầu người dùng chạy SQL thủ công.

---

[SECURITY]

Không in:

- API Key
- Password
- Secret

Không hardcode thông tin nhạy cảm.

---

[OUTPUT]

Hiển thị:

- Danh sách package đã cài
- Kết quả import từng package
- Python interpreter đang sử dụng
- Trạng thái ChromaDB
  - Server hoặc
  - Embedded Local
- Trạng thái PostgreSQL
- Trạng thái database rag\_db
- Những việc người dùng cần thực hiện (nếu có)

Không tạo code RAG ở bước này.

## Bước 4 — Xây dựng RAG

Gửi prompt sau cho AI Agent:

[CONTEXT] Đọc agent spec tại file SPEC\_bui\_06.md

[INPUT]

Đọc JSON tại: `RAG/rag\_foundation/bui\_05/output/chunks/`  
(Không đọc file khác).

[GOAL]

Viết toàn bộ logic trong file `rag.py`. Chỉ cần các hàm sau:

1. `index()`
  - Đọc JSON
  - Tạo embedding với Gemini với số chiều là 384 để đồng bộ với fallback của chromadb minilm-l6-v2
  - Lưu text vào PostgreSQL/ Nếu ko tìm thấy PostgreSQL được khởi động, lưu ra disk local với tên .db
  - Lưu embedding vào ChromaDB
2. `ask(question)`
  - Embedding câu hỏi với Gemini với số chiều là 384 để đồng bộ với fallback của chromadb minilm-l6-v2
  - Tìm top-k, chuyển tham số k
  - Lấy text tương ứng từ PostgreSQL/ Nếu ko tìm thấy PostgreSQL được khởi động, đọc ra disk local với tên .db
  - Gửi cho Gemini
  - Trả lời
3. `status()`
  - Số lượng document
  - Số lượng chunk

Ràng buộc:

- Nếu thiếu `GEMINI\_API\_KEY`: Vẫn cho phép retrieval, không gọi LLM.
- Không retry, không batch, không logging, không dùng framework phức tạp.

- Ưu tiên code đơn giản.
- Sử dụng: `google-genai`, model `gemini-embedding-2` và `gemini-flash-lite-latest`.

---

## Bước 5 — Tạo giao diện

Gửi prompt sau cho AI Agent:

```
[CONTEXT] Đọc agent spec tại file SPEC_bui_06.md
[GOAL]

Fill vào `app.py` với Streamlit. Giao diện chỉ cần:

Sidebar:

- Trạng thái PostgreSQL
- Trạng thái ChromaDB
- Trạng thái Gemini API Key (Có / Thiếu)

Main Area:

- Nút Index
- Ô nhập câu hỏi
- Kết quả Top-k
- Answer (Câu trả lời)

(Không tạo: login, history, dashboard, analytics).

Pipeline xử lý:
`Question` → `Top-k` → `Gemini` → `Answer`

Lưu ý: Nếu thiếu API Key, chỉ hiển thị Retrieval, không gọi Gemini.
```

---

## 6. Tiêu chí hoàn thành (Checklist)

- ☐ Sử dụng đúng `.venv` của Buổi 5.
- ☐ Chỉ đọc JSON trong thư mục `output/chunks`.
- ☐ PostgreSQL lưu trữ nội dung và metadata.
- ☐ ChromaDB lưu trữ vector embedding.
- ☐ Gemini dùng thư viện `google-genai` với model `gemini-embedding-2` và `gemini-flash-lite-latest`.
- ☐ Streamlit hiển thị được danh sách top-k và câu trả lời.
- ☐ Bảo mật: Không lộ API Key hoặc password trong logs/code.
- ☐ Tổng số dòng mã nguồn toàn project khoảng **300–500 dòng Python**.